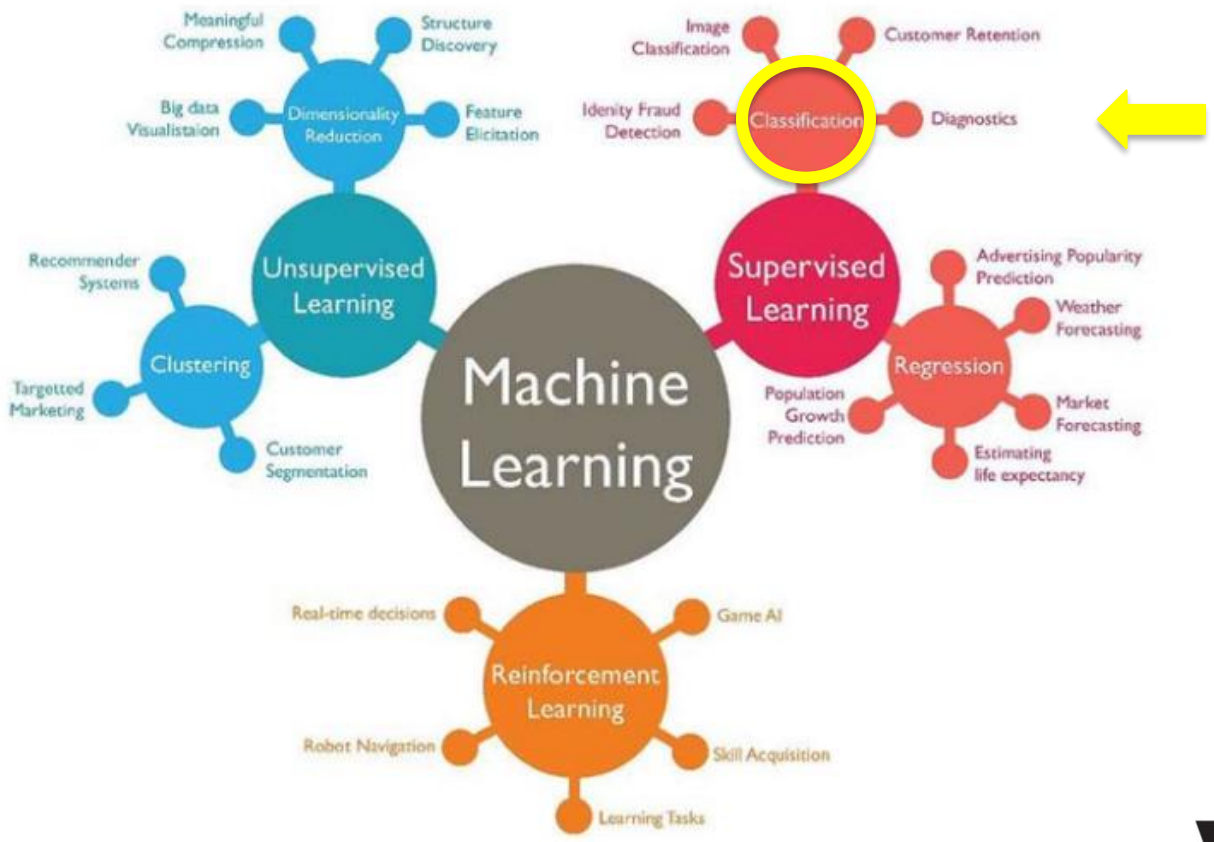
Machine Learning Fundamentals

LINEAR CLASSIFICATION

Andy Louwyck
andy.louwyck@vives.be
Academic year 2025 - 2026

hogeschool vives

The 3 big families



Category of machine learning. Image by https://www.techleer.com/articles/203-machine-learning-algorithm-backbone-of-emerging-technologies/

hogeschool vives

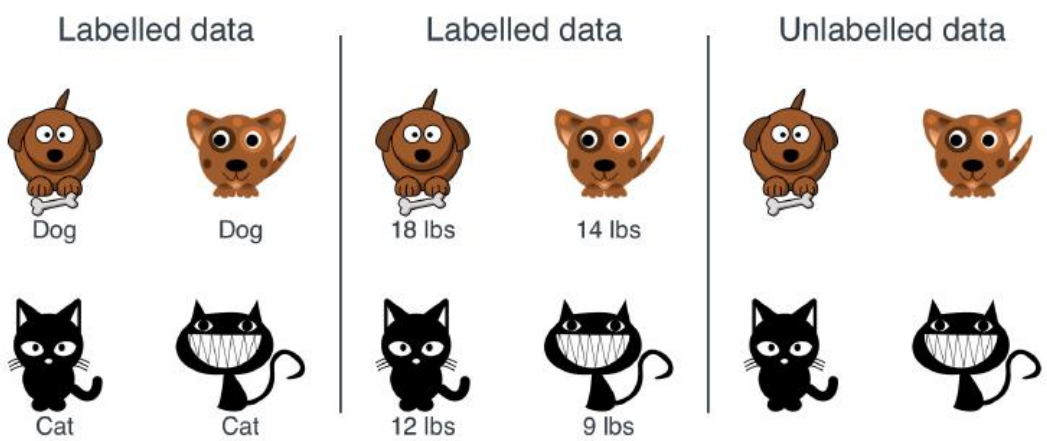
Labeled and unlabeled data

- Labeled data: data with a label

  → **SUPERVISED LEARNING**

- Unlabeled data: data without a label

  → **UNSUPERVISED LEARNING**



Data, examples, features, and target

- **Data** = information = table
- **Example** = sample = instance = data point = table row/record
- **Feature** = independent variable = table column/attribute
- **Target** = labels = dependent variable = feature we want to predict

data(set)

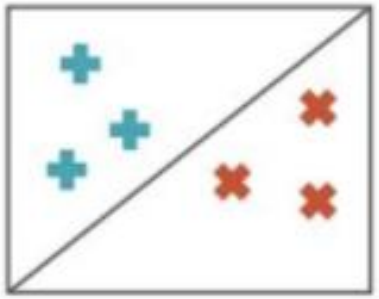
	A	B	C	D	E	F
1	id	date	size	typos	recipients	spam
2	0	12/01/2021	2.5	0	1	False
3	1	13/01/2021	1.3	0	2	False
4	2	14/01/2021	12.1	3	15	True
5	3	15/01/2021	7.8	2	19	True
6	4	16/01/2021	4.6	1	5	False
7	5	17/01/2021	9.8	5	1	True
8	6	18/01/2021	11.6	3	63	True

example

feature **target**

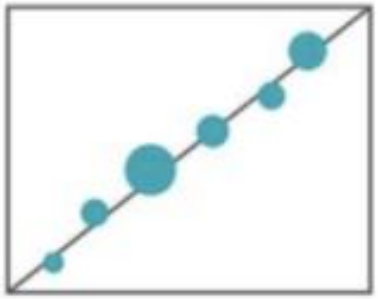
Classification and regression

- **Classification:**
  - Models predicting a **categorical variable**
  - Model output is discrete: a category, a class, a state, …
  - Examples: spam or ham, images with cat or dog, MNIST numbers
- **Regression:**
  - Models predicting a **numerical variable**
  - Model output is (usually) continuous: a real number
  - Examples: predicting house prices, linear regression



CLASSIFICATION

Sorting items into categories



REGRESSION

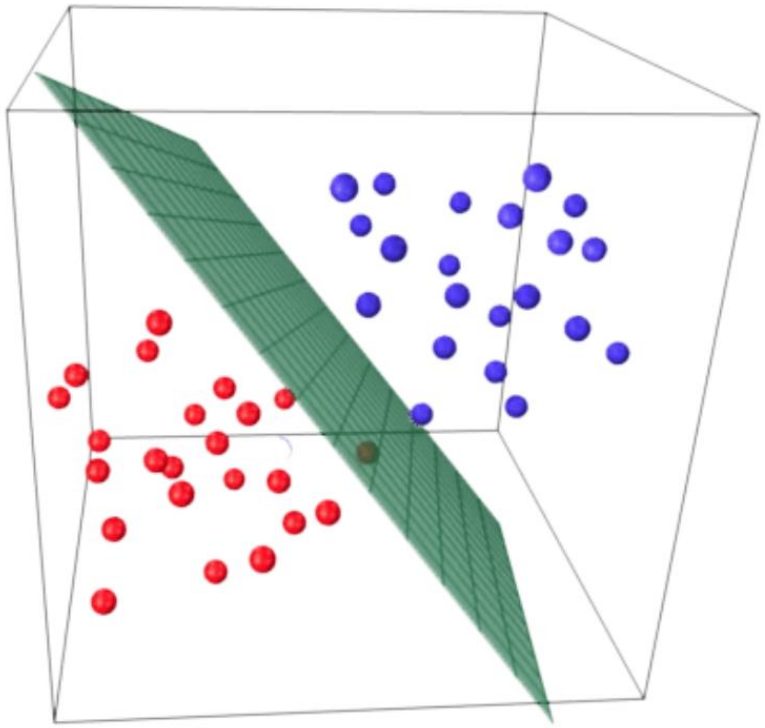
Identifying real values (dollars, weight, etc.)

Linear classification

Separates classes with a **straight decision boundary:**

straight line in 2D, plane in 3D, or hyperplane in ND.

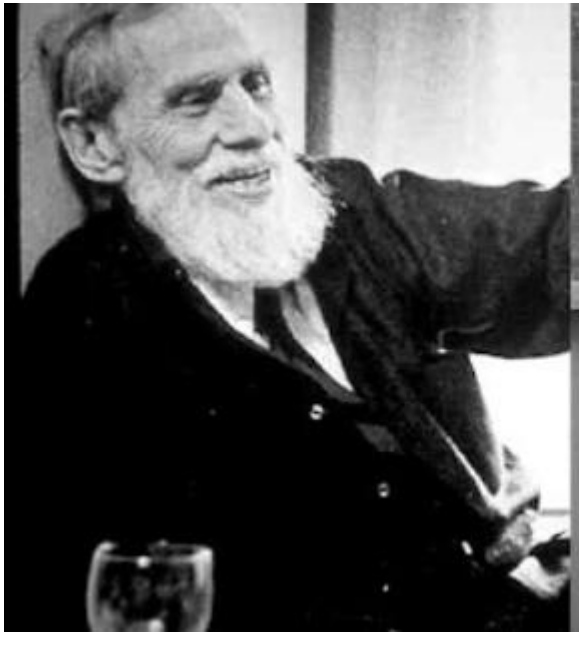
- **Perceptron**: Simple binary rule that updates weights on mistakes.
- **Logistic regression**: Linear model with sigmoid output for probabilities.
- **Support Vector Machine**: Finds the widest-margin separating hyperplane.

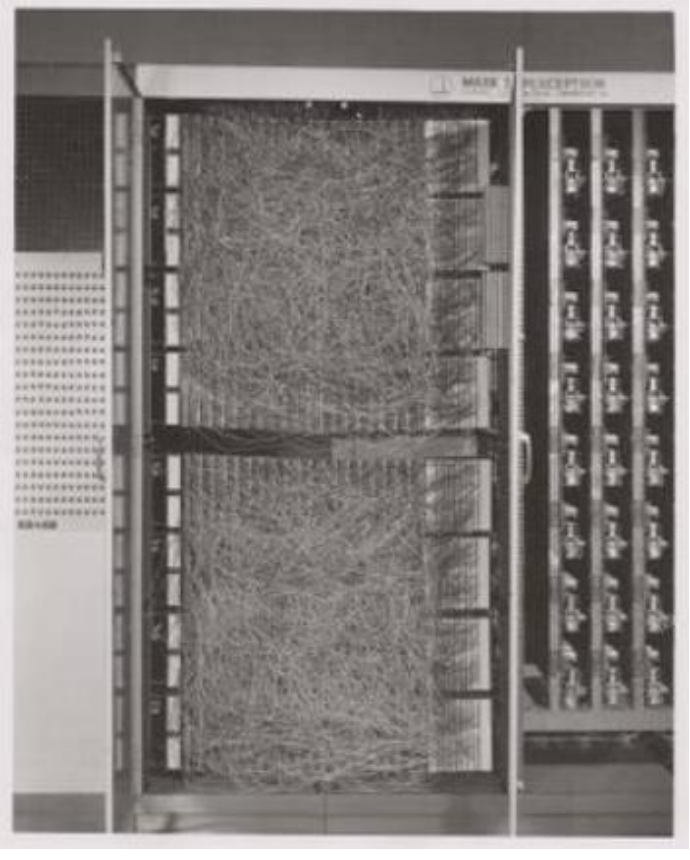


A little bit of history

The perceptron algorithm:

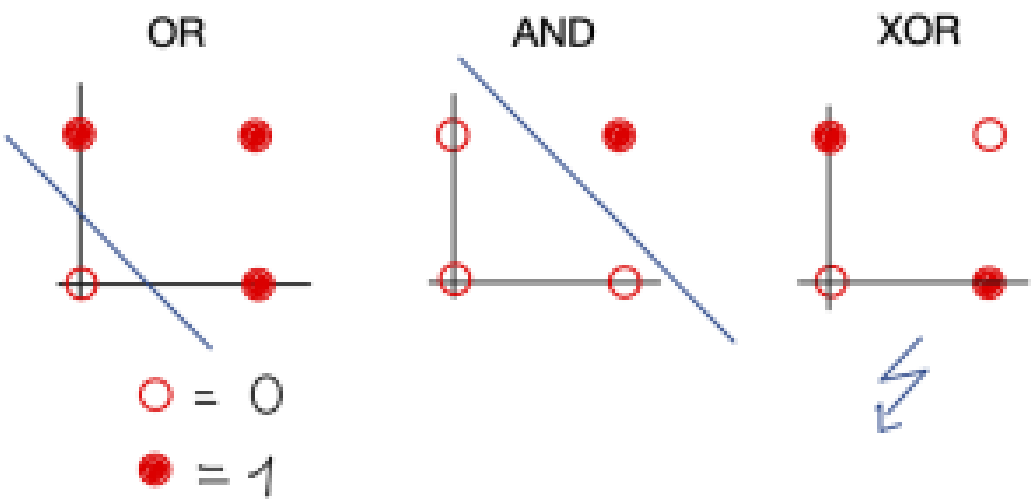
- Developed in 1957 by Rosenblatt
- Based on the McCulloch-Pitts model of a neuron from 1943

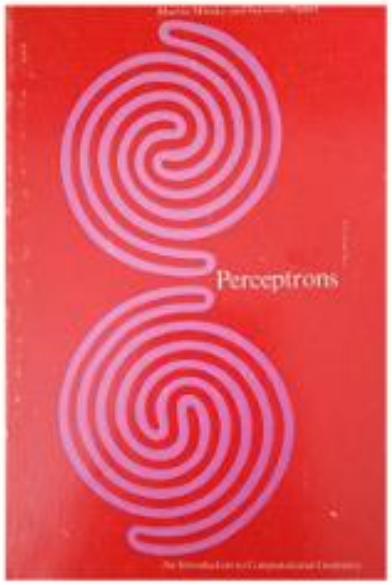


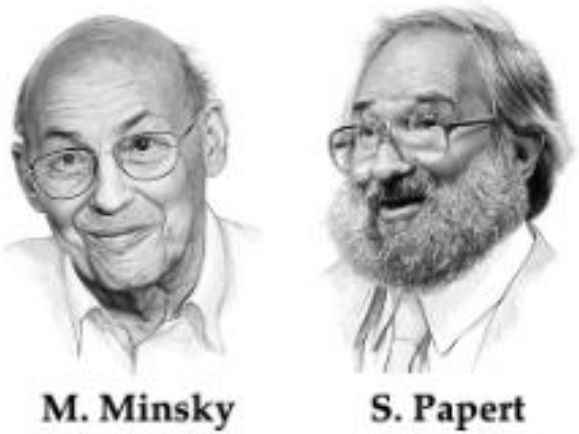


Criticism

- Minsky & Papert book on perceptrons (1969)
- Rosenblatt's perceptron cannot implement the XOR gate
- Pessimistic in its predictions for AI
- A possible cause for the first AI winter in the 1970s





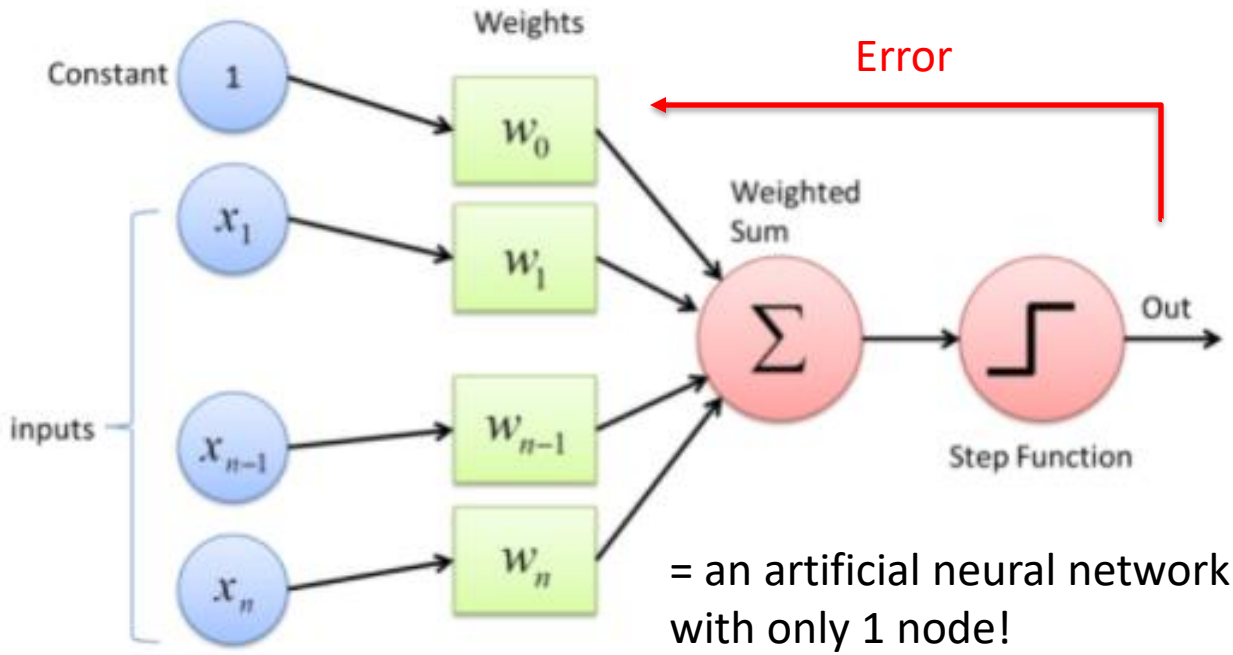


Perceptron

A perceptron is often represented schematically in this way:

- data points "inputs" with features X_1, X_2, ...
- parameters or weights w_1, w_2, ...
- intercept or bias b is represented here as w_0
- weighted sum = sum(X*w) + b = dot(X, w) + b
- Heaviside step function as activation
- predictions "out" = 0 or 1
- perceptron loss function

At the end of the lesson everything will be clear!!



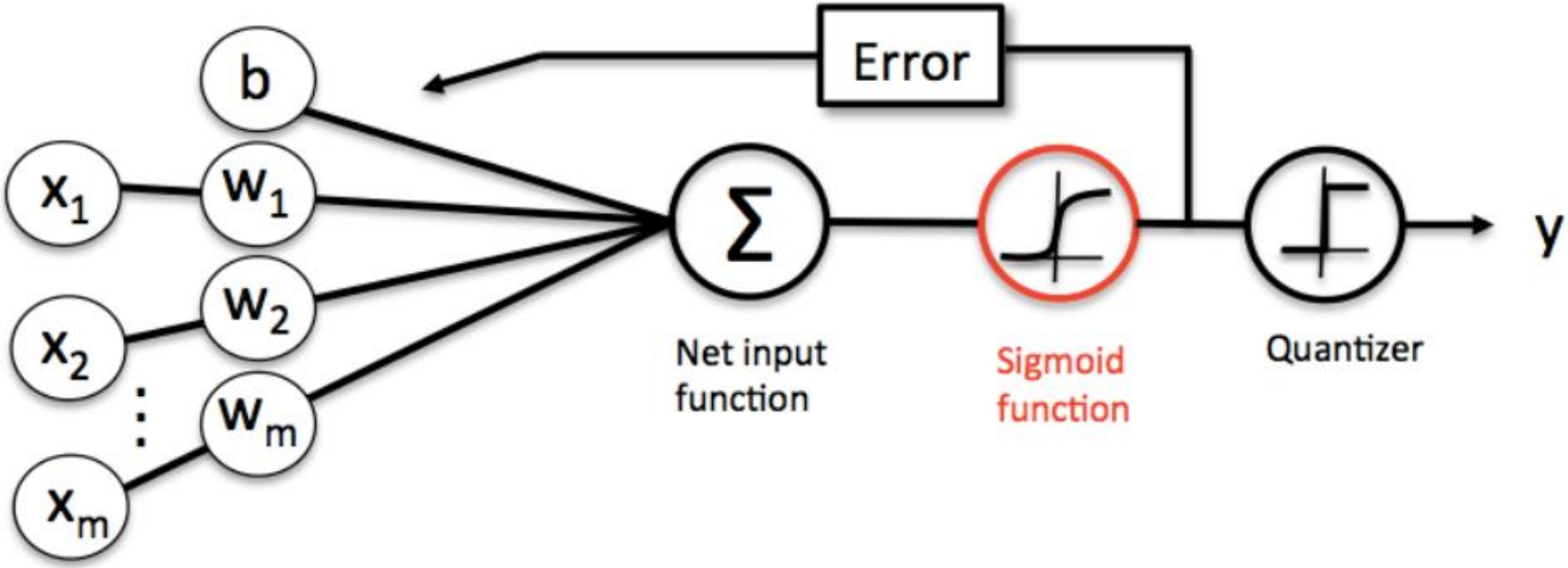
= an artificial neural network with only 1 node!

Logistic Regression

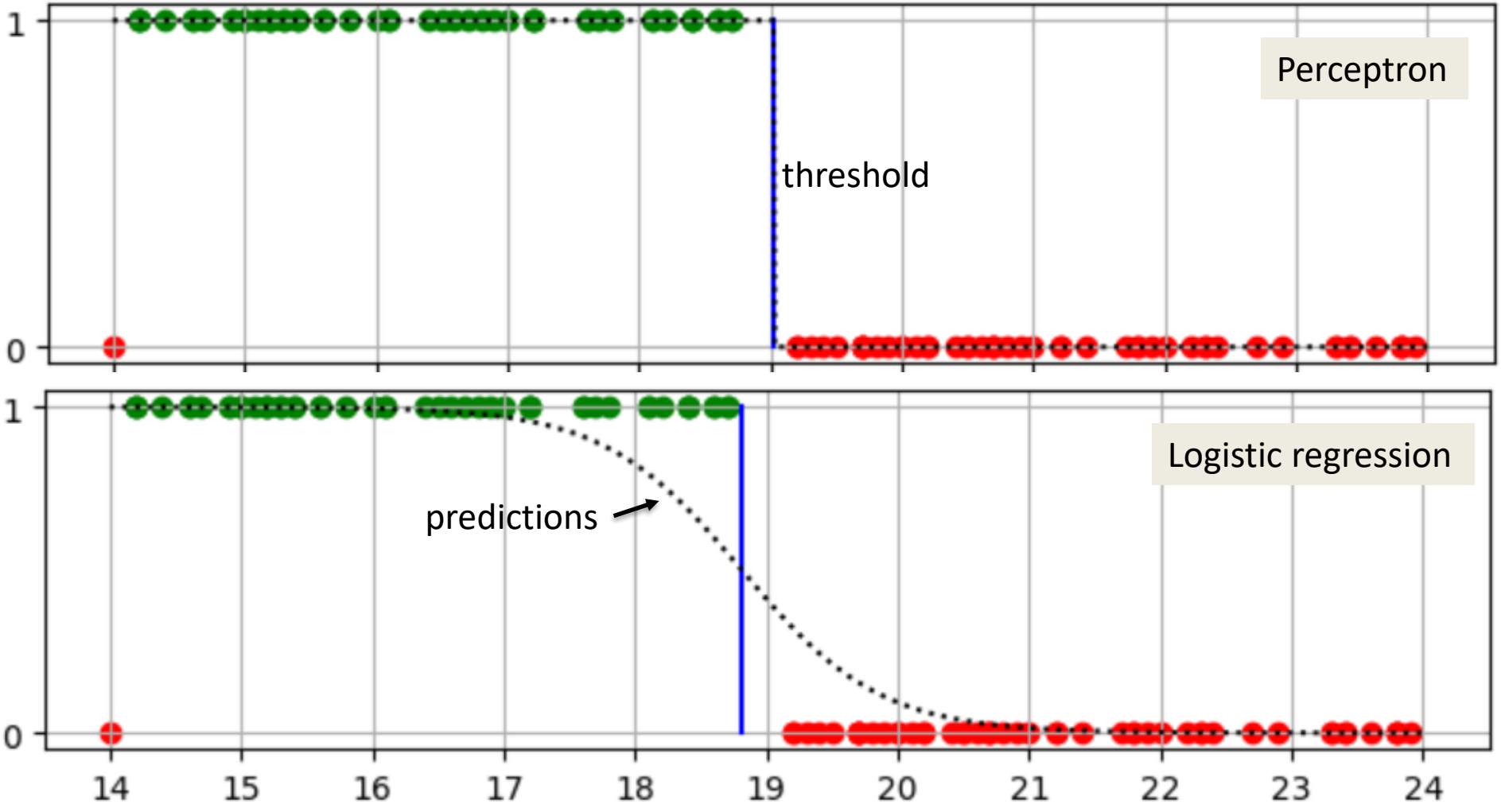
Logistic Regression = continuous perceptron:

- Output between 0 and 1 (= probabilities!)
- Other activation function: sigmoid function
- Other loss function: log loss = binary cross-entropy

For the rest: same algorithm as perceptron algorithm!



Thermostat dataset



How these models make predictions

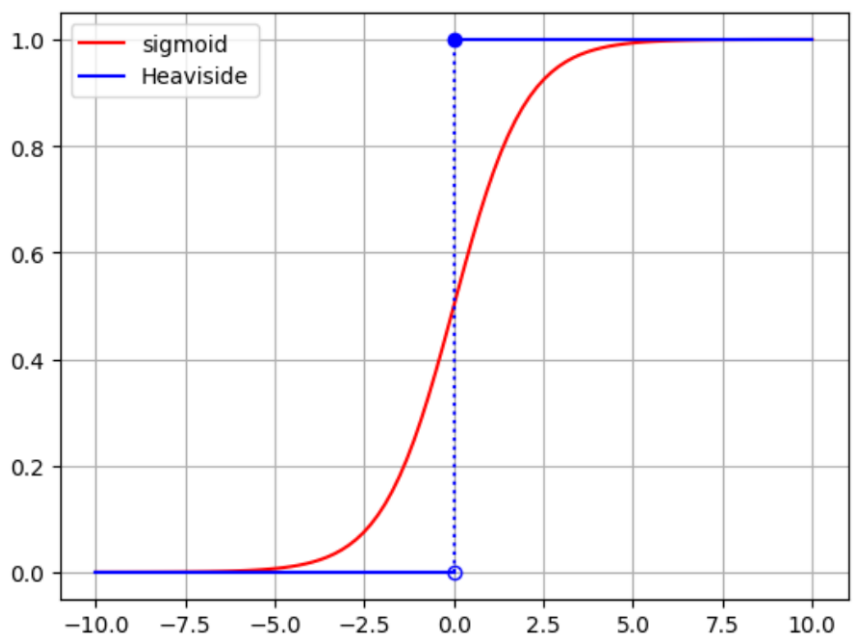
Perceptron

1. $z_i = wX_i + b$
2. $\hat{y}_i = H(z_i)$

Logistic Regression

1. $z_i = wX_i + b$
2. $\hat{y}_i = \sigma(z_i)$
3. $round(\hat{y}_i)$

$$H(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

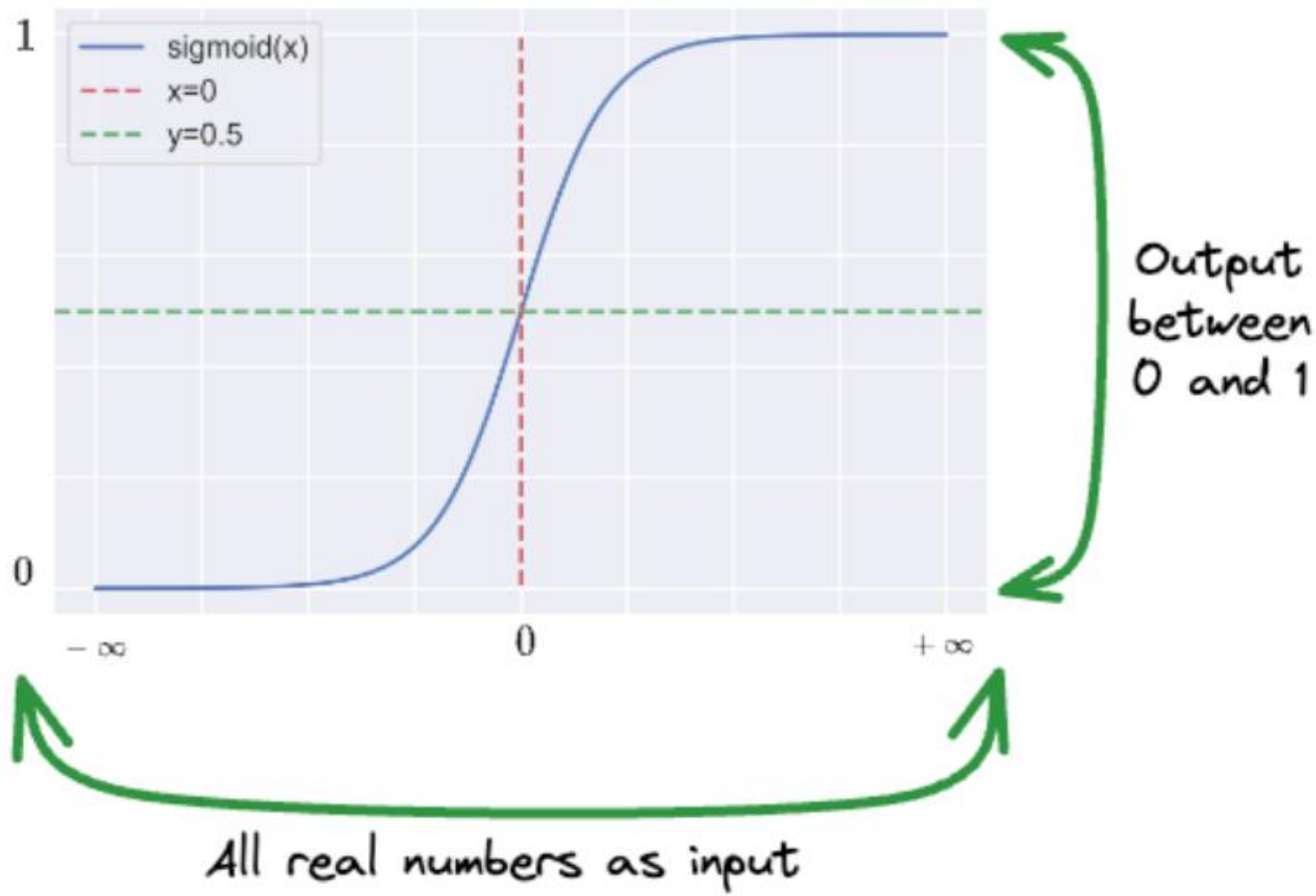
z_i are scores or **logits**

Heaviside H and sigmoid σ are **activation functions**

Sigmoid function σ

The sigmoid function outputs the probability that the label equals 1:

- $\sigma(-\infty)$: 0%
- $\sigma(0)$: 50%
- $\sigma(+\infty)$: 100%



How these models define loss

- **Perceptron loss**

$$L = \sum_{i=1}^{n} (\hat{y}_i - y_i) z_i$$

- **Binary cross-entropy** or log loss (logistic regression)

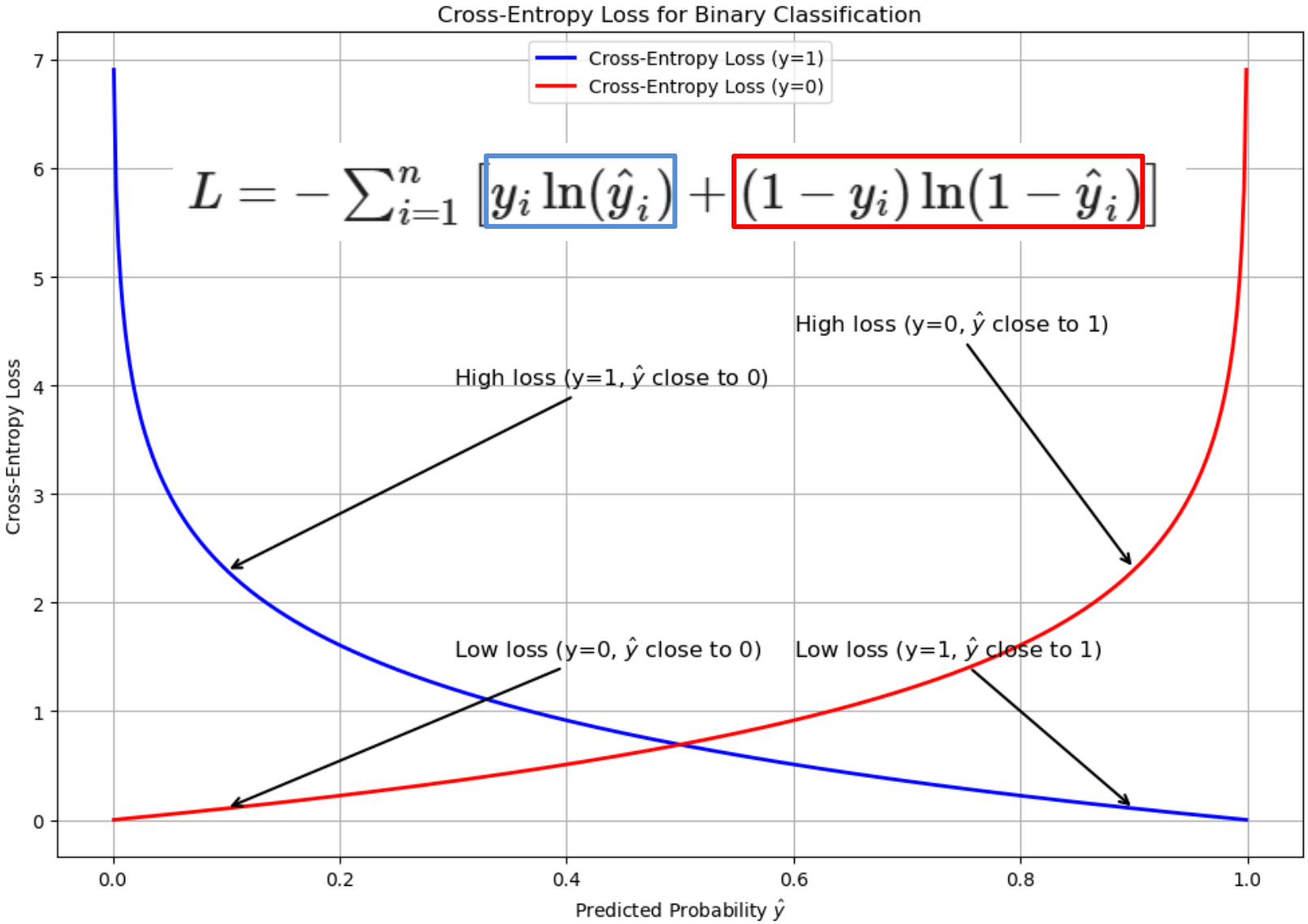
$$L = -\sum_{i=1}^{n} \left[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right]$$

- **Hinge loss** (SVM)

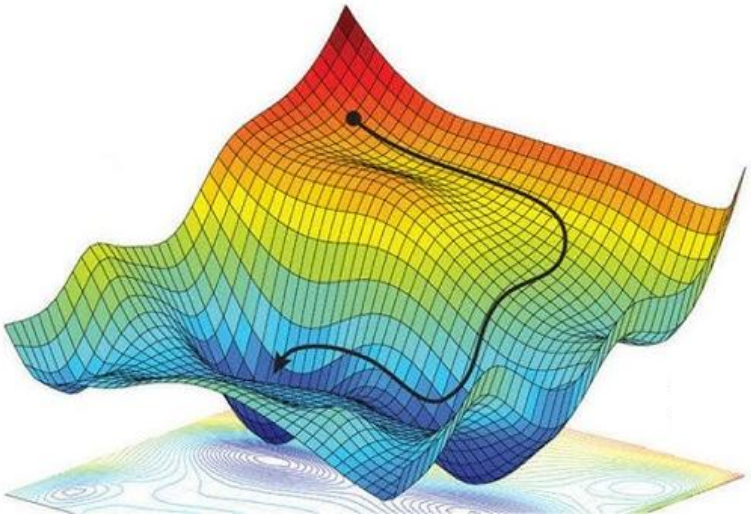
$$L = \sum_i \max(0, 1 - y_i z_i)$$

Note: the final value taken for reporting is almost always a **mean** which is obtained by dividing the sum by the number of samples *n*

Binary Cross-Entropy



Stochastic Gradient Descent



Given:

- a dataset with n data points (x_i, y_i)
- the number of epochs n_{epochs}
- the learning rate α

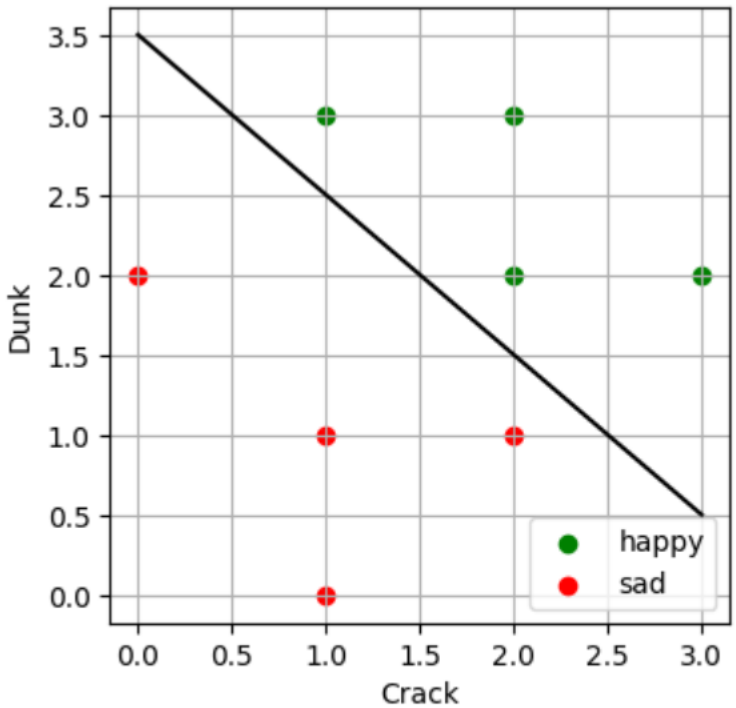
> Steps:
>
> 0. Initialize parameters a and b with random values
> 1. Repeat n_{epochs} times:
> 2. Repeat n times:
> 3. Select a random data point (x_i, y_i) from the dataset (without replacement!)
> 4. Compute the prediction $\hat{y}_i = A(ax_i + b)$, with A the activation function (Heaviside or sigmoid)
> 5. Compute the deviation $\epsilon_i = y_i - \hat{y}_i$, and multiply the deviation by the learning rate: $\delta_i = \alpha\epsilon_i$
> 6. Adjust parameters a and b by means of the deviation:
>
> - $a_{new} = a + \alpha(y_i - \hat{y}_i)x_i = a + \delta_i x_i$
> - $b_{new} = b + \alpha(y_i - \hat{y}_i) = b + \delta_i$

Linear classification in 2 dimensions

- 2 features X_1 and X_2
- 1 binary target **y**
- Decision boundary: straight line separates the data points

	Sentence	Crack	Dunk	Mood
0	Crack!	1	0	sad
1	Dunk dunk!	0	2	sad
2	Crack dunk!	1	1	sad
3	Crack crack dunk!	2	1	sad
4	Crack dunk dunk dunk!	1	3	happy
5	Crack dunk dunk crack!	2	2	happy
6	Crack dunk crack dunk crack!	3	2	happy
7	Crack dunk dunk crack dunk!	2	3	happy

X y

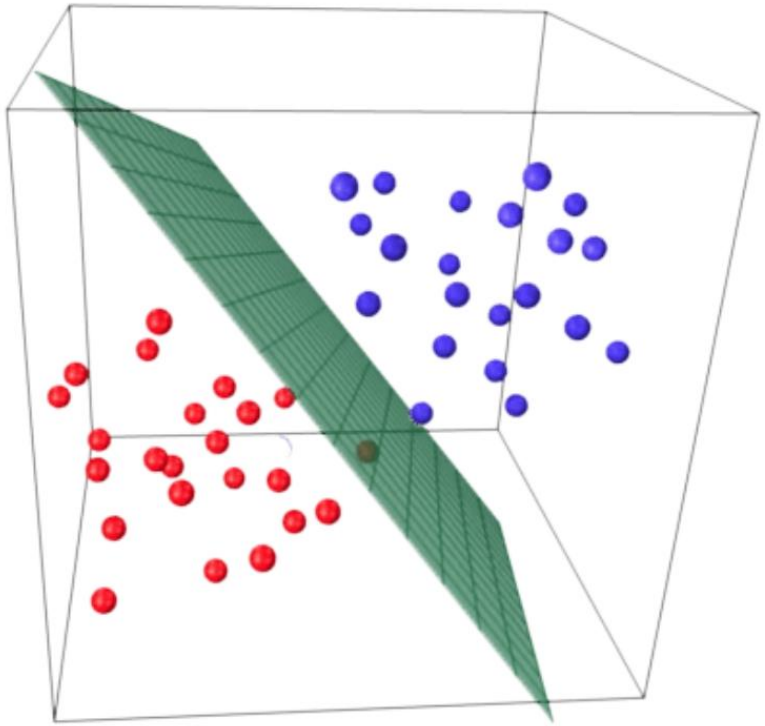


Linear classification in N dimensions

- Dot product: $\mathbf{z} = b + \mathbf{X}\mathbf{w}$

```
w = np.array([w1, w2])  # create vector of weights
z = b + np.dot(X, w)    # alternatively, you can write b + X @ w
```

- Decision boundary: hyperplane



How to determine the decision boundary?

Solve: $\boldsymbol{z = b + \sum_j w_j X_j = 0}$

- **1 feature**: decision boundary is a point (threshold)

$$b + wX = 0 \Longrightarrow X = -b/w$$

- **2 features**: decision boundary is a straight line (1D)

$$b + w_1 X_1 + w_2 X_2 = 0 \Longrightarrow X_2 = -(b + w_1 X_1)/w_2$$

- **3 features**: decision boundary is a plane (2D)
- **N+1 features**: decision boundary is a hyperplane (ND)

Scikit-Learn classes

- **Perceptron and logistic regression**

```
from sklearn.linear_model import Perceptron
from sklearn.linear_model import LogisticRegression
```

- **Normalizing features**

```
from sklearn.preprocessing import MinMaxScaler
```

$$X_j^s = \frac{X_j - \min(X_j)}{\max(X_j) - \min(X_j)}$$

```
from sklearn.preprocessing import StandardScaler
```

$$X_j^s = \frac{X_j - \mathrm{mean}(X_j)}{\mathrm{std}(X_j)}$$

Scikit-Learn solvers

- **Perceptron**
  - Stochastic Gradient Descent (no explicit “solver” input)
  - Uses `SGDClassifier` class
  - “learning rate” and “maximum number of iterations”

- **LogisticRegression**
  - lbfgs: default solver
  - liblinear: small datasets, simple regularization
  - newton-cg: medium-size data, smooth optimization
  - sag / saga: large datasets, faster on many samples

- Both support **regularization** (“penalty”)

Example: Breast Cancer dataset

- Load dataset

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer(as_frame=True)  # "as_frame=True" returns the dataset as Pandas DataFrame
df = data.frame  # the dataframe is stored in attribute "frame"
X = df[df.columns[:-1]]  # features
y = df[df.columns[-1]]   # target
```

- Scale features

```
scaler = MinMaxScaler()  # instantiate
X_scaled = X.copy()      # perform scaling on a copy of X
X_scaled[features_to_scale] = scaler.fit_transform(X[features_to_scale])
```

- Train and evaluate models

```
sk_perceptron = Perceptron(eta0=0.1)  # instantiate with learning rate set to 0.1
sk_perceptron.fit(X_scaled, y)  # train
print(f'accuracy is {sk_perceptron.score(X_scaled, y):.4f}')  # accuracy
```

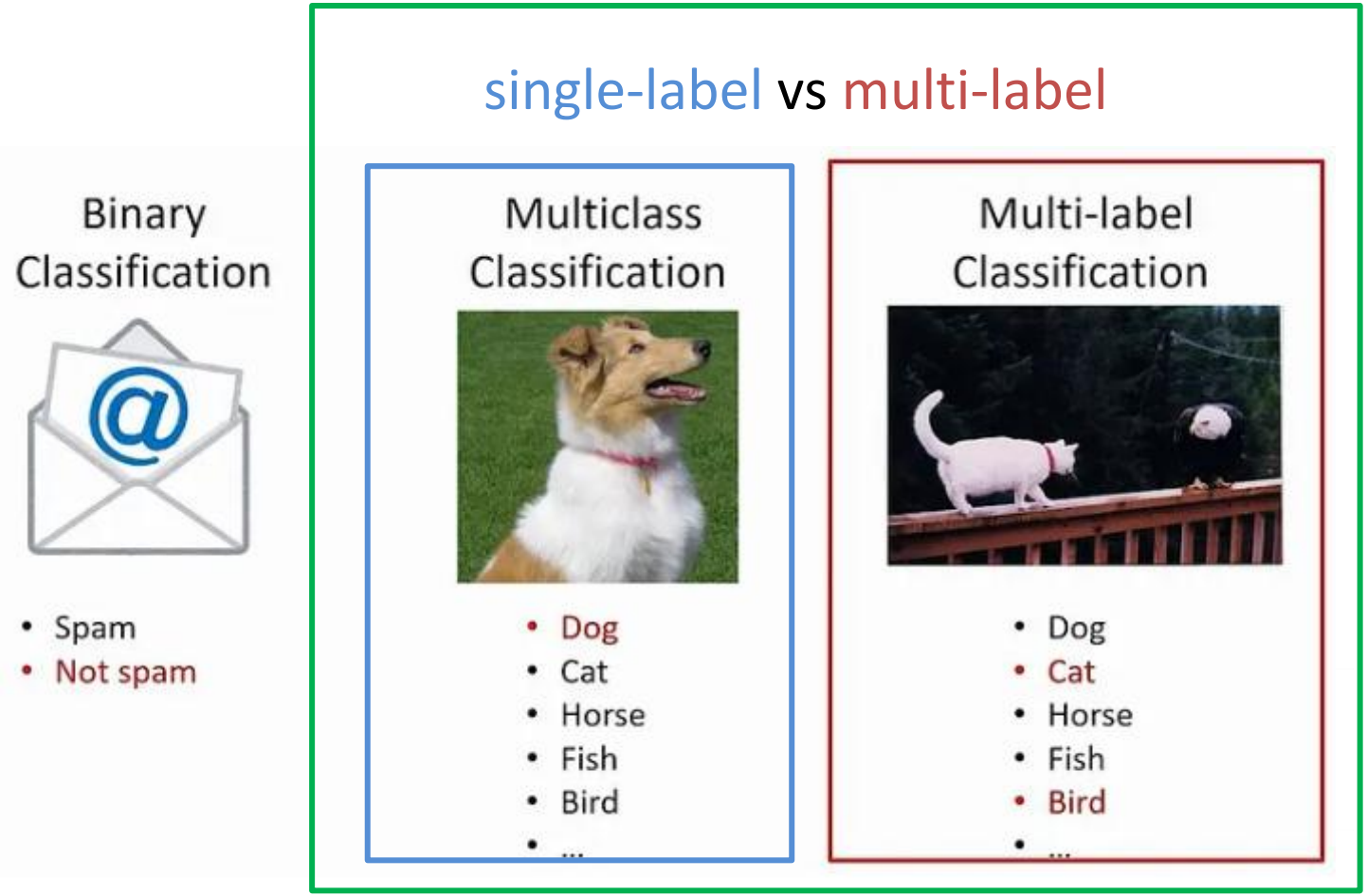
```
accuracy is 0.9631
```

```
sk_logistic_regression = LogisticRegression(penalty=None)  # instantiate without regularization
sk_logistic_regression.fit(X_scaled, y)  # train
print(f'accuracy is {sk_logistic_regression.score(X_scaled, y):.4f}')  # accuracy
```

```
accuracy is 0.9859
```

Multinomial classification

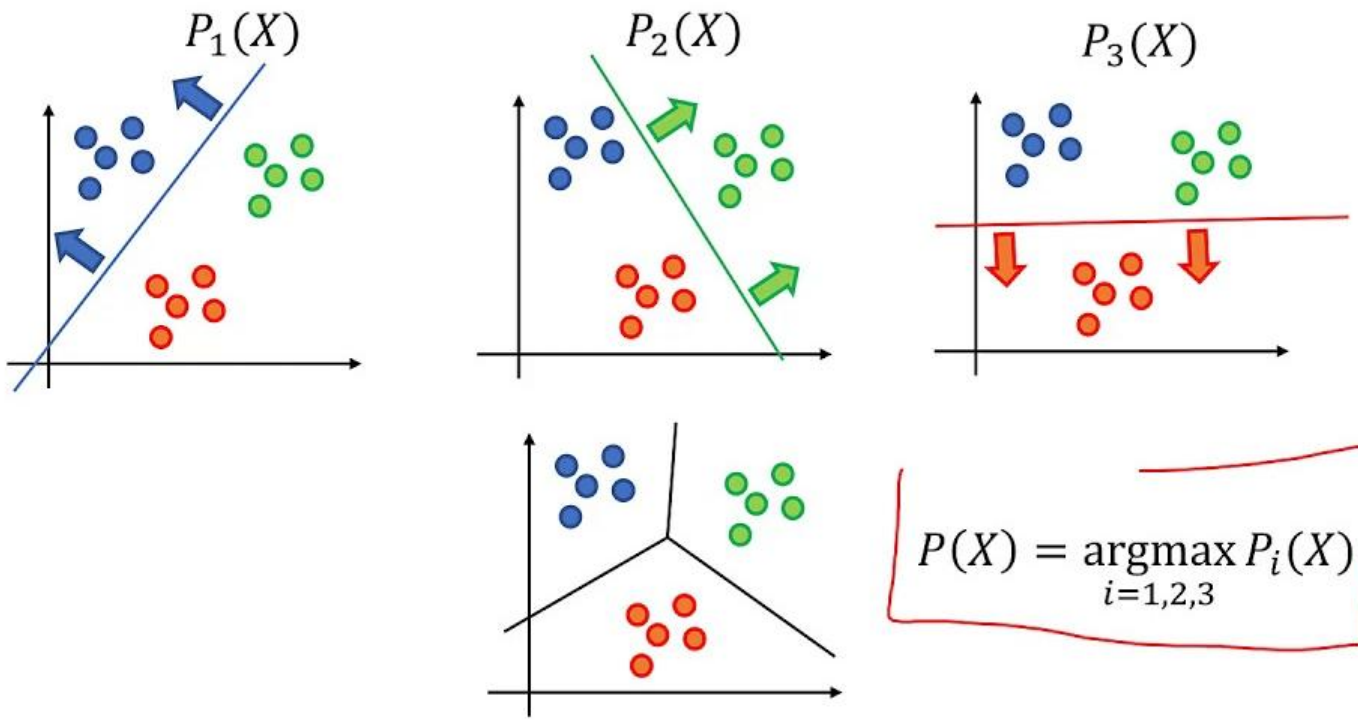
= multiclass classification



One versus Rest

- K classes: train K binary classifiers

  E.g.: K logistic regression models
- K probabilities as output: select class with the highest probability



$$P(X) = \underset{i=1,2,3}{\operatorname{argmax}} P_i(X)$$

Multinomial Logistic Regression

- Uses **softmax** as activation (= generalization of sigmoid):

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K$$

- $\mathbf{z}$: input vector (raw scores / logits) of length K.
- z_i: the i-th component of $\mathbf{z}$.
- K: number of classes.
- $\sigma(\mathbf{z})_i$: softmax output for class i, interpreted as the probability that the class is i.

- Uses **cross-entropy** as loss (= generalization of binary cross-entropy)

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

Example: Iris dataset (3 classes)

- Load dataset

```
from sklearn.datasets import load_iris
iris = load_iris()  # by default, as_frame is False, returning numpy arrays instead of a dataframe
X = iris.data       # 4 features as numpy array
y = iris.target     # target as numpy array with class labels 0, 1, and 2
```

- One vs Rest classifier

```
from sklearn.multiclass import OneVsRestClassifier

# pass a LogisticRegression instance to the OneVsRestClassifier class
OvR = OneVsRestClassifier(LogisticRegression(penalty=None))
OvR.fit(X, y)
OvR_acc = OvR.score(X, y)
print(f"One vs Rest accuracy: {OvR_acc:.4f}")
```

```
One vs Rest accuracy: 0.9800
```

- Multinomial logistic regression

```
# the LogisticRegression class automatically applies the softmax function if more than 2 classes
SR = LogisticRegression(penalty=None)
SR = SR.fit(X, y)
SR_acc = SR.score(X, y)
print(f"Softmax Regression accuracy: {SR_acc:.4f}")
```

```
Softmax Regression accuracy: 0.9867
```